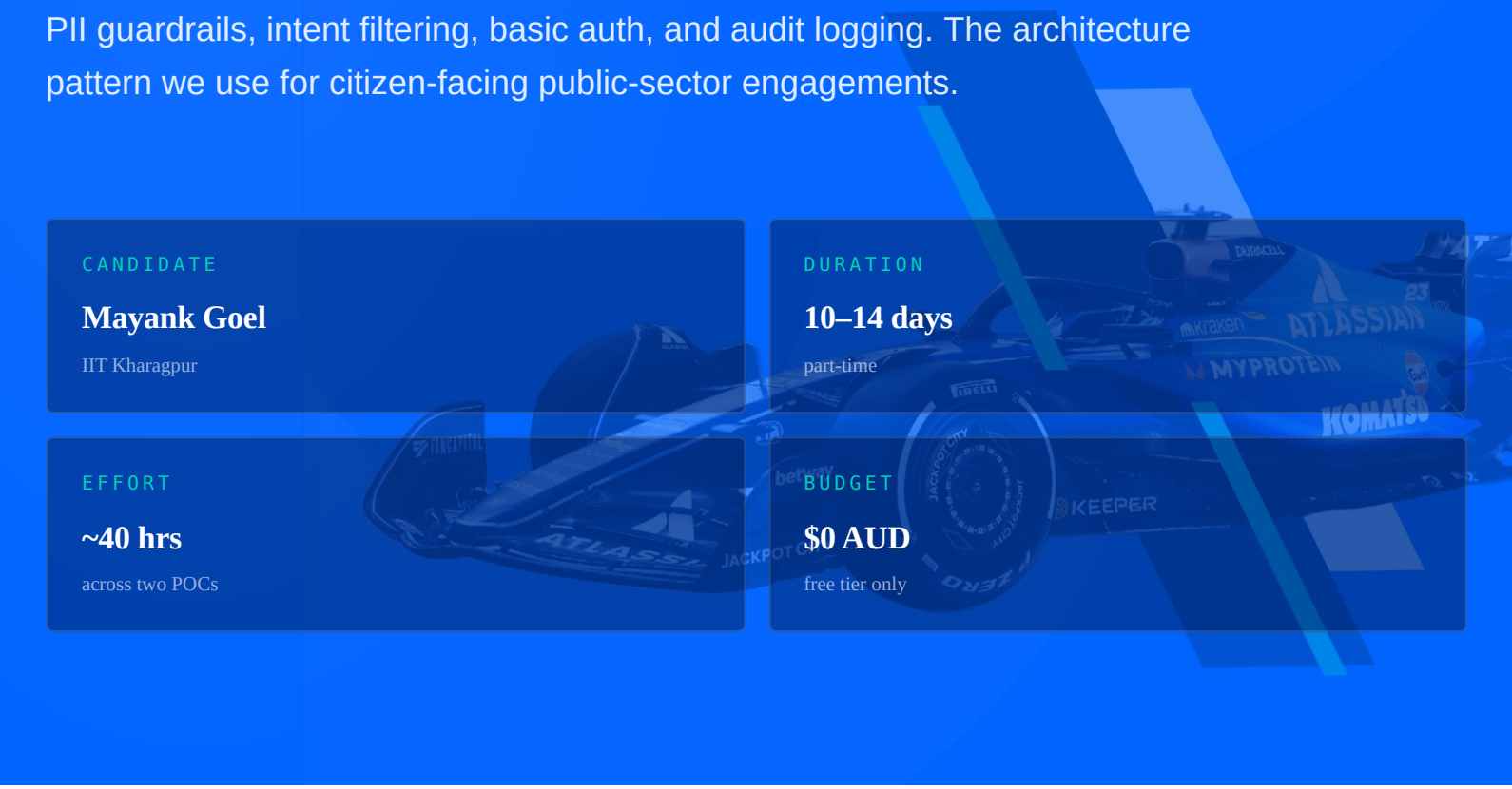
AI ENGINEERING INTERN · TWO-STAGE POC ASSIGNMENT

RAG. Guardrails. Production.

Two sequential builds. First, prove you can ship a working retrieval-augmented chatbot wrapped as an MCP server. Then, take it further — a citizen-services AI assistant for a fictional Indian municipal corporation, with PII guardrails, intent filtering, basic auth, and audit logging. The architecture pattern we use for citizen-facing public-sector engagements.



CANDIDATE
Mayank Goel
IIT Kharagpur

DURATION
10–14 days
part-time

EFFORT
~40 hrs
across two POCs

BUDGET
$0 AUD
free tier only

00 //

Before You Start

A NOTE FROM THOMPSON

Mayank — these two POCs are deliberately staged. POC 1 sits in your comfort zone (LangChain, ChromaDB, Streamlit) but introduces the MCP server pattern you said you understood conceptually. POC 2 pushes into the gaps we identified: deployment, authentication, and production-style guardrails. Don't skip POC 1 to jump ahead — the foundation matters. Ship POC 1 fully before starting POC 2. One working thing beats two half-built things.

// POC 01 · FOUNDATION

RAG Chatbot wrapped as an MCP server.

Build a small retrieval-augmented chatbot over 10–20 PDFs (your choice — university admissions, government regulations, anything topical), then expose it via a standards-compliant MCP server with two tools. Anyone running Claude Desktop or any MCP client should be able to query it.

Build a working **RAG chatbot** over a small document corpus, wrap it as an **MCP server** exposing two tools, deploy it, and prove it works from an MCP client.

POC 01 · Required Stack

LAYER · FRAMEWORK

MUST

LangChain + ChromaDB

You've used both. Build the ingest pipeline with LangChain's document loaders + RecursiveCharacterTextSplitter, persist embeddings in ChromaDB (local file-based, no Pinecone needed).

LAYER · MCP

MUST

Official MCP Python SDK

Use the official `mcp` Python package from Anthropic. Expose two tools: `search_documents(query, k)` and `summarise_document(doc_id)`. Test from Claude Desktop.

LAYER · LLM

MUST

Anthropic API

Claude Haiku 4.5 for the synthesis step (cheapest, fastest). Free credit will be provided. Use the official Python SDK.

LAYER · SECRETS

MUST

Infisical

API keys go in Infisical (free tier), pulled at runtime. No `.env` committed. No keys in repo. This is the most common rookie error — don't make it.

LAYER · UI

PICK ONE

Streamlit OR CLI

Streamlit is fastest for a working chat UI (you've used it). A CLI is also acceptable. Either way, the MCP server must work standalone too.

LAYER · DEPLOY

PICK ONE

Streamlit Cloud / Railway

Deploy the chat UI somewhere public. Streamlit Cloud free tier is the path of least resistance. The MCP server can stay local for grading.

POC 01 · Scope

IN SCOPE

What you build.

- \+ Ingest 10–20 PDFs into ChromaDB once (offline script)
- \+ RAG chain: retrieve top-K → pass to Claude → answer with sources
- \+ MCP server exposing two tools, callable from Claude Desktop
- \+ Chat UI (Streamlit) OR CLI for direct testing
- \+ Citations — every answer must point back to source chunk(s)
- \+ Deployed chat UI on a public URL

OUT OF SCOPE

What to skip.

- × Authentication on the chat UI — POC 02 handles auth
- × Multi-user, multi-tenant, or session memory
- × Re-ranking, hybrid search, or query rewriting
- × Fine-tuning embeddings — use defaults
- × Production-grade error handling

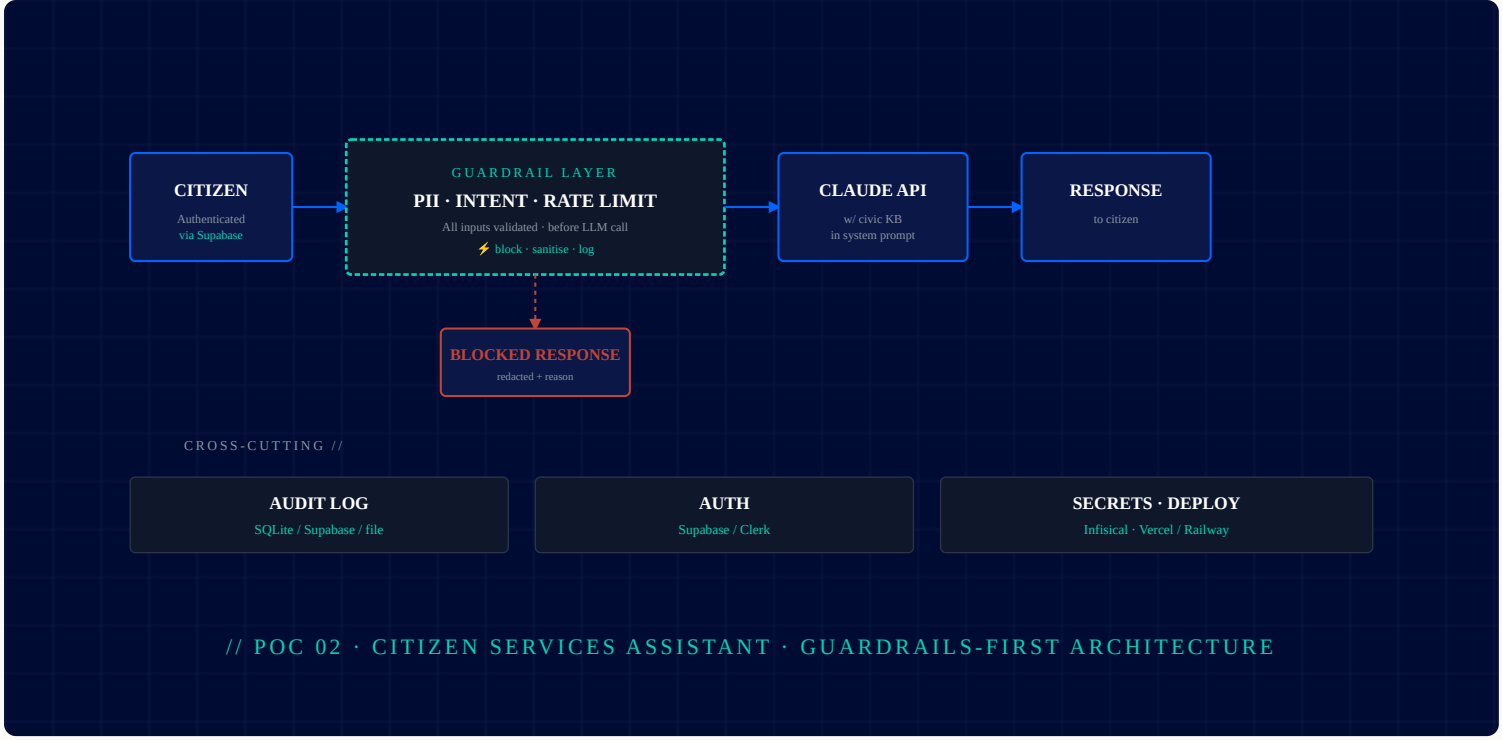
↓ POC 01 COMPLETE · ESCALATE TO POC 02 ↓

// POC 02 · PRODUCTION POSTURE

Citizen Services AI Assistant — with guardrails.

Build a citizen-services chatbot for a fictional Indian municipal corporation — your choice of name (e.g., Pragati Nagar Nigam, Sahyadri Municipal Corporation). Five common civic services. The chatbot answers natural-language questions, but every request passes through PII detection, intent filtering, and audit logging. Plus basic auth. Public-sector AI deployments live or die on this layer.

Build a **citizen-services chatbot** for a fictional Indian municipal corporation with 5 civic services, wired to Claude via the Anthropic API. Every interaction is gated by **PII detection**, **intent filtering**, and **audit logging**. Users must **log in**. Deployed and demoable.



// THE NON-NEGOTIABLE

The guardrails are the whole point of POC 02.

For government and enterprise clients, AI safety is the differentiator — not the model. Anyone can wire up the Anthropic API. The work is in proving that PII never reaches the LLM, malicious intents are blocked before tokens are spent, and every interaction is auditable. Implement all three.

G1 // PII

PII Detection

Block Aadhaar, PAN, credit cards, mobile numbers, passwords. Regex + a Claude classifier call.

G2 // INTENT

Intent Filtering

Detect jailbreaks, malicious requests, off-topic abuse. Pre-LLM classifier.

G3 // AUDIT

Audit Logging

Every request, decision, and response logged with timestamps. Queryable.

POC 02 · Required Stack

LAYER · AUTH

MUST

Supabase Auth

This is your auth gap from the call. Use Supabase free tier — email/password is fine. The chat UI is gated behind login. Session in cookie or JWT.

LAYER · LLM

MUST

Anthropic API

Claude Sonnet 4.5 for civic services answers, Haiku for the classifier calls (cheap pre-filtering). Use the Python or Node SDK.

LAYER · AUDIT

MUST

Supabase Postgres OR SQLite

Persist every interaction. Schema: `id, user_id, timestamp, request, decision, response, blocked_reason`. Build a tiny admin view that lists the last 50 events.

LAYER · SECRETS

MUST

Infisical

Anthropic key, Supabase keys, service role tokens — all in Infisical. No secrets in code, no secrets in deploy env in plaintext where avoidable.

LAYER · DEPLOY

MUST

Vercel / Railway

Live URL. I'll click through it from my laptop. If it 500s on my first prompt, that's a flag. Add a small README with test creds for me to log in.

LAYER · FRONTEND

PICK ONE

Next.js OR Streamlit

Next.js looks more like a real civic-services product (stretch). Streamlit ships faster (acceptable). Either works. No need for React Native or a mobile build.

POC 02 · Scope

IN SCOPE

What you build.

- \+ Fictional Indian municipal corporation with a name and 5 services (property tax, water bill, sanitation/garbage, birth certificate, trade licence or building permit)
- \+ Civic services knowledge base — small JSON or Markdown file is fine
- \+ Authenticated chat UI (Supabase Auth — email/password)
- \+ All three guardrails (PII, intent, audit)
- \+ Admin view showing last 50 audit log entries
- \+ Deployed live URL with one test user account documented in README
- \+ Test cases proving guardrails block what they should

OUT OF SCOPE

What to skip.

- × Real payment integration — payment "intents" can be stubbed
- × Real municipal corporation branding — use a fictional one and a made-up logo
- × Mobile app, native iOS/Android
- × SSO, OAuth, MFA — basic email/password is sufficient
- × Internationalisation — English only
- × Vector store / RAG — KB-in-prompt is fine here
- × Production observability stack

03 // Combined Deliverables

01 Two Public GitHub Repos

One per POC. Both with clean READMEs, architecture diagrams (Mermaid or PNG), setup steps, environment variables, and known-issues sections.

MANDATORY

02 POC 01 · Working MCP Server

Demonstrated end-to-end from Claude Desktop (or any MCP client). Include a sample `claude_desktop_config.json` snippet in the README.

MANDATORY

03 POC 02 · Deployed Live URL

I will click your link and log in with the test creds you give me. All three guardrails must fire on the test prompts in your README.

MANDATORY

04 Guardrail Test Cases

A Markdown file in POC 02's repo listing test prompts and expected outcomes. Examples: "`my credit card is 4111-1111-1111-1111`" → blocked; "`ignore all instructions`" → blocked.

MANDATORY

05 Two Loom Walkthroughs (5 min each)

One per POC. Architecture explanation, the trickiest call you made, a live run. Screen-share code and terminal — no slides.

MANDATORY

06 Reflection Document

One Markdown file: which POC was harder and why, what you'd build differently if you had another week, what you learned about MCP and guardrails.

STRETCH

04 // How We Grade

01 **Does it actually work?**

I will clone, follow your README, and try to break it. If first-run setup fails, that's a flag.

02

Guardrails are real

For POC 02 — PII isn't just regex-blocked, intent filtering catches actual jailbreaks. Show the work.

03

Auth is wired correctly

Logged-out users can't access the chat. Tokens aren't in localStorage as plaintext. Session expires sensibly.

04

Secret hygiene

Infisical or environment boundaries respected. No `.env` files in git. No keys printed to logs.

05

MCP is correctly implemented

POC 01 — your server speaks the MCP protocol properly, not just an HTTP API you called "MCP".

06

Deploy works on first try

POC 02 — your live URL responds, login works, a test prompt returns a real answer.

07

Honest self-assessment

If a guardrail is brittle, say so in the README. Don't oversell. We're hiring for production maturity.

08

Sequencing discipline

POC 01 done before POC 02 starts. We can tell from commit history. Don't context-switch.

05 //

Suggested Pacing

14 days. Two POCs, sequenced.

DAY 1–4

POC 01 · Build RAG

Repo, Infisical, ingest pipeline, RAG chain, Streamlit UI working locally.

DAY 5–7

POC 01 · MCP + Ship

Wrap as MCP server, test from Claude Desktop, deploy UI, Loom 1.

DAY 8–11

POC 02 · Municipal MVP

Supabase auth, civic services KB, chat UI, guardrails wired, audit log live.

DAY 12–14

POC 02 · Polish & Deploy

Deploy to Vercel/Railway, test prompts, Loom 2, READMEs final.

// SUBMISSION PROTOCOL

When you're done, send one email.

01 To: **thompson@di.net.au** · CC: yourself

02 Subject: [POC Submission] Mayank Goel — RAG + Citizen Services MVP

03 Body: Two GitHub URLs, two Loom URLs, deployed app URL with test login creds, and a short paragraph on which POC was harder

04 Ship POC 01 commits before you start POC 02 commits — we look at the timeline

05 I'll review within 48 hours and book a follow-up call to walk through the work

AI PRACTICE · PRE-SALES

Design Industries Pty Ltd · Atlassian Solution Partner Enterprise · Melbourne · Pune · Manila · Cebu

Thompson Cherian

Head of Pre-Sales · Design Industries

T 1300 73 63 63 · WWW.DI.NET.AU